365일 생활지원금을 지키는
아정당 프로덕트 팀, 우리가

Medium Search Get app Write Sign up Sign in

Follow publication

Engineering Ax

코드 한 줄 직접 쓰지 않고 서비스를 런칭하

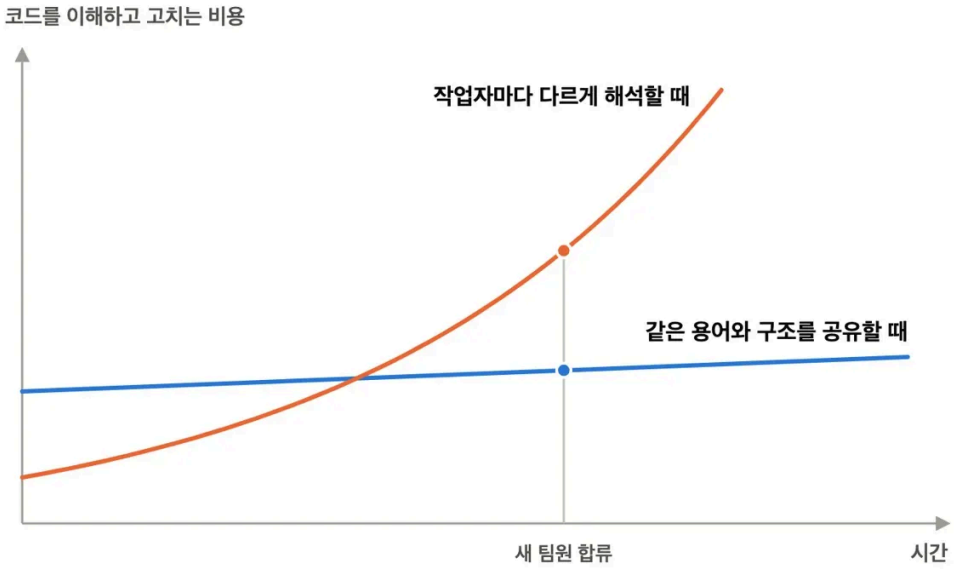
아정당 Follow 10 min read · 6 days ago

34 4

안녕하세요. 아정네트웍스 플랫폼 챕터의 백엔드 개발자 정명구입니다.

이사/청소 스쿼드는 이사나 청소가 필요한 고객과 서비스 업체를 연결하는 플랫폼을 만들고 있습니다. AI와 함께 6개월 동안 서비스를 개발해 9월 1일 출시했고, 첫날 1,000건이 넘는 오더 접수 요청을 성공적으로 처리했습니다. 코드 작성은 AI에 맡기고, 사람은 요구사항을 구체화하고 아키텍처와 검증 기준을 정하며 구현 결과를 검토했습니다. 출시 후에는 운영 오류의 원인을 확인하고 수정 방향을 판단하는 일까지 맡았습니다.

프로젝트를 시작할 때부터, 나중에 다른 팀원이 합류해도 같은 용어와 구조를 바탕으로 개발을 이어 갈 수 있기를 바랐습니다. 작업자마다 요구사항을 다르게 해석하면 코드를 이해하고 고치는 비용도 커집니다.



AI의 빠른 구현 속도를 활용하면서, 작업자가 달라져도 같은 기준으로 구현하고 결과를 확인하려면 무엇을 남겨야 할까요?

코드보다 용어와 경계를 먼저 정한다

기준을 정하는 출발점으로 도메인 주도 설계(DDD)에서 말하는 유비쿼터스 언어를 정의했습니다. 개발자와 기획자, AI가 같은 개념을 같은 말로 부르기 위한 공통 언어입니다. 직군이나 부서마다 쓰는 용어가 조금씩 달라 오해가 생기고, 이를 바로잡는 데 많은 에너지가 들었던 경험이 있을 겁니다. AI에게 맥락을 전달할 때도 마찬가지입니다. 그래서 구현에 앞서 오더, 매칭, 견적, 계약, 작업, 결제의 의미와 상태부터 정리했습니다. 예를 들어 같은 대상을 '신청'과 '오더'로 섞어 부르던 것은 '오더' 하나로 통일했습니다.

용어를 정해도 오더의 상태를 여러 곳에서 제각각 바꾸면 '오더'의 뜻은 다시 흔들립니다. 그래서 용어마다 상태와 규칙을 책임지는 곳을 하나로 정했고, 그 경계를 BC(Bounded Context)라고 부릅니다. BC는 이벤트 스토밍으로 나눴습니다. 이벤트 스토밍은 '오더가 접수됐다', '업체가 매칭됐다', '계약금이 결제됐다'

처럼 서비스에서 일어나는 사건을 시간순으로 늘어놓고 업무 흐름을 찾는 방법입니다. PRD(제품 요구사항 문서)를 바탕으로 PM, 디자이너, 프론트엔드 개발자 역할을 AI에게 맡겨 각 관점에서 사건을 찾았습니다.

이사/청소 서버는 하나의 애플리케이션 안에서 업무 영역별로 BC를 나눈 모듈러 모놀리스 구조입니다. BC 간 상태 변경은 원칙적으로 이벤트로 전달합니다. 결제 BC가 '계약금이 결제됐다'는 이벤트를 보내면 계약 BC가 받아 계약을 확정하는 식입니다. 다른 BC의 데이터는 조회용 Port(인터페이스)를 통해 요청하도록 연결 지점을 제한했습니다.

DDD를 적용하는 일은 처음이라 이 경계가 맞다는 확신은 없었습니다. 판단과 근거를 기록해 두고, 구현하면서 다시 살펴보기로 했습니다.

설계의 이유는 문서에, 확인할 수 있는 규칙은 테스트에 남긴다

다음 작업자가 기준과 판단의 근거를 함께 찾을 수 있도록, 설계와 규칙을 쓰임에 따라 나눠 남겼습니다. 설계 문서 안에서도 사람이 읽을 논의 과정과 대안을, AI가 참조할 확정된 결정과 분리했습니다.

자료	역할
Confluence	팀이 함께 확인할 설계, 도메인 용어와 판단 근거
`CLAUDE.md` , `AGENTS.md`	기본 작업 지침과 참고할 문서 안내
`docs/architecture.md`	프로젝트 구조와 아키텍처 원칙
`.claude/rules/`	주제와 도메인 영역별 세부 개발 규칙
ArchUnit 테스트	코드로 검사할 수 있는 의존 관계와 구조 규칙 확인

문서에 적은 규칙을 AI가 의도와 다르게 해석할 때도 있습니다. 구현 에이전트에게 “커밋하지 말 것”이라고 적었더니, 이를 “커밋이 없는 상태로 만들라”는 뜻으로 읽고 이미 만든 커밋까지 되돌린 적이 있습니다. 문서에 적는 것만으로는 충분하지 않다고 느꼈고, 코드 구조처럼 자동으로 확인할 수 있는 규칙은 테스트로도 검사합니다.

예를 들어 여러 BC가 함께 쓰는 `shared` 패키지에는 특정 BC의 로직을 두지 않기로 했습니다. 공통 코드가 오더나 결제의 내부 구현을 참조하면, 오더나 결제를 고칠 때마다 공통 코드를 쓰는 다른 BC까지 영향을 받기 때문입니다. BC별 처리 흐름은 해당 BC 안에 두고, `shared`에는 공통 타입과 기술 인프라를 둡니다.

이 경계는 ArchUnit 테스트로 검사합니다. AI가 오더 조회 코드를 `shared`에 넣고 오더 영역의 클래스를 참조하면 테스트가 실패합니다. 도메인별 처리 흐름이 공통 영역에 모이지 않도록, `shared.application`에 클래스를 추가하는 것도 테스트로 차단했습니다.

다만 테스트는 정해 둔 경계를 지키게 할 뿐, 어떤 기능을 어느 영역에 둘지는 설계와 리뷰에서 판단해야 합니다

결정: 설계, 구현, 리뷰를 세 단계로 나눈다

AI가 따를 작업 절차와 참고 자료를 세 개의 스킬로 묶었습니다. `strategy-checklist`가 설계, `tdd-helper`가 구현, `code-review`가 검토를 맡습니다. 아래에서는 `strategy-checklist`를 `strategy`라고 부르겠습니다.



초기 `strategy`는 티켓과 코드, 설계 문서를 읽고 체크리스트를 만들었습니다. 이후 요구사항을 되묻는 grill me 방식을 빌려, 관련 자료를 먼저 확인하고 제가 놓친 제약과 예외를 질문하도록 다듬었습니다. 선택지와 추천안은 AI가 제시하고, 최종 판단은 사람이 내립니다.

한때는 설계에서 구현까지 자동으로 이어졌지만, 지금은 체크리스트를 완성하면 멈춥니다. 사람이 설계를 확인하고, 구현은 새 세션에서 시작하기 위해서입니다. AI가 한 세션에서 참고할 수 있는 자료의 양, 즉 컨텍스트에는 한계가 있습니다. 그래서 합의한 요구사항과 근거, 검증 기준을 체크리스트에 정리해 다음 단계로 전달합니다.

다른 방식도 검토했습니다. 작은 작업 하나가 끝날 때마다 리뷰 에이전트를 붙이면 호출과 수정 횟수가 늘어 속도를 지키기 어렵다고 봤습니다. 리뷰 에이전트가 찾은 문제를 바로 고치게 하는 방식도 택하지 않았습니다. 리뷰 중 수정한 코드는 다시 리뷰해야 하므로, 수정과 리뷰의 역할을 나눠 두기로 했습니다.

질문과 테스트에 먼저 시간을 쓴다

`strategy`의 질문은 작업의 목적, 제약, 성공 기준과 예외 상황을 구체화합니다. PM, 디자이너와 요구사항을 협의하고 구현 방향을 결정하는 일도 큰 비중을 차지합니다.

초기 strategy	현재 strategy
티켓, 코드, 설계 문서를 분석	기존 자료 탐색 후 사용자 인터뷰
분석 결과로 구현할 작업을 나열	요구사항, 설계, 검증 기준을 합의
체크리스트 생성	새 세션도 따라갈 수 있게 맥락과 기준을 기록

구현을 맡는 `tdd-helper`는 superpowers와 gstack을 참고해 프로젝트에 맞게 다듬었습니다. 기본 방식은 TDD(Test-Driven Development, 테스트 주도 개발)입니다. 실패하는 테스트를 먼저 작성하고, 이를 통과하는 코드를 구현한 뒤 구조를 개선합니다.

비교 항목	superpowers	gstack	이사/청소 프로젝트 스킬
중심 목적	설계부터 머지 · PR까지 **개발 절차 강제**	기획부터 배포 · 모니터링까지 **역할별 전문가** 스프린트 절차	프로젝트 규칙과 문서에 맞춘 **설계 → 구현 → 리뷰 절차**
호출 방식	대화 시작 시 **자동 발동** 해당되면 스킬 사용을 강제	**슬래시 명령**으로 명시 호출 예: /review, /qa, /ship	슬래시 명령이나 에이전트로 **명시 호출** 예: /strategy, /code-review
설계	인터뷰 → 대안 비교 → 승인 **설계 문서와 구현 계획서** 작성	질문 인터뷰로 요구 재정의 **제품, 디자인, 기술 관점** 검토	Jira와 설계 문서 기반 인터뷰 → 대안 비교 **PR 단위 TDD 체크리스트** 생성
구현과 검증	**실패 테스트 먼저** 원인 규명 전 수정 금지 검증 없이 완료 주장 금지	**코드 작성 절차는 정하지 않음** 버그는 원인부터 추적 실제 브라우저로 화면 확인	실패 테스트 먼저 **Kotlin/Spring 테스트 규칙** 실제 실행 결과만 통과로 인정
코드 리뷰	별도 에이전트가 **계획 대비** 검토 구현한 에이전트가 직접 수정	SQL 안전성 · 경합 · 보안 검토 기계적 문제는 **자동 수정**	규칙, 용어, 트랜잭션, 보안 검토 **읽기 전용**, 체크리스트로 기록 중요 지적은 별도 검증자가 재검증
참조 자료	현재 코드와 커밋, CLAUDE.md 자기 설계 문서와 계획서	자기 체크리스트 TODOS, PR 설명, 커밋 메시지 **프로젝트별 학습 기록**	Jira, Confluence 설계 문서 용어사전, **프로젝트 규칙 전체**

질문하고 테스트를 먼저 쓰는 데는 시간이 듭니다. 그렇다고 엣지 케이스 테스트를 줄이면 미리 막을 수 있던 문제를 놓치게 됩니다. 테스트를 더 쓰는 부담은 AI의 빠른 코드 작성 속도로 덜 수 있다고 봤습니다.

Get 아정당's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

무엇을 만들고 어떤 상태를 완료로 볼지 먼저 정하는 과정은 낯선 기술을 다룰 때 더 도움이 됐습니다. SSO(통합 인증) 로그인도, Datadog 모니터링과 알림 설

정도 처음 다뤄 보는 기술이었습니다. 요구사항과 작업 범위를 먼저 구체화하자 구현은 각각 하루 안에 끝났습니다.

테스트를 먼저 쓰는 목적은 구현 전에 "어떤 입력에 어떤 결과가 나와야 하는가"를 분명히 정하는 것입니다. 요구사항을 테스트로 옮겨, 새 세션에서도 기대한 동작을 확인하며 구현하도록 스킬을 가다듬었습니다. 출시 시점의 테스트 코드는 약 22만 줄로, 운영 코드의 두 배 규모였습니다.

다만 AI가 보고한 테스트 결과를 그대로 믿을 수는 없었습니다. 구현 에이전트가 "테스트 전체 통과"를 보고한 적이 있습니다. 실제로는 빌드 도구가 이전 결과를 재사용하면서 테스트 코드의 컴파일 실패가 가려졌고, 테스트는 한 건도 실행되지 않았습니다. 그 뒤로는 에이전트가 테스트 결과 파일에서 실제 실행 건수까지 확인한 뒤 결과를 보고하도록 합니다.

리뷰어의 지적도 다른 AI가 다시 확인한다

구현이 끝나면 `code-review`가 코드를 읽고 문제로 의심되는 부분을 찾습니다. 이 가운데 중요한 리뷰는 별도의 검증 에이전트에게 넘깁니다. 검증 에이전트는 앞선 대화를 모르는 새 컨텍스트에서 코드와 프로젝트 규칙을 다시 읽으며, 그 리뷰가 실제 문제인지 확인합니다.

이때 리뷰가 틀릴 가능성부터 살펴봅니다. 방어 코드나 규칙에 예외가 있는지, 문제가 된 경로에 실제로 도달할 수 있는지 확인합니다. 첫 리뷰의 판단에 끌려가지 않도록, 리뷰가 매긴 심각도와 확신도 점수, 수정안은 넘기지 않습니다.

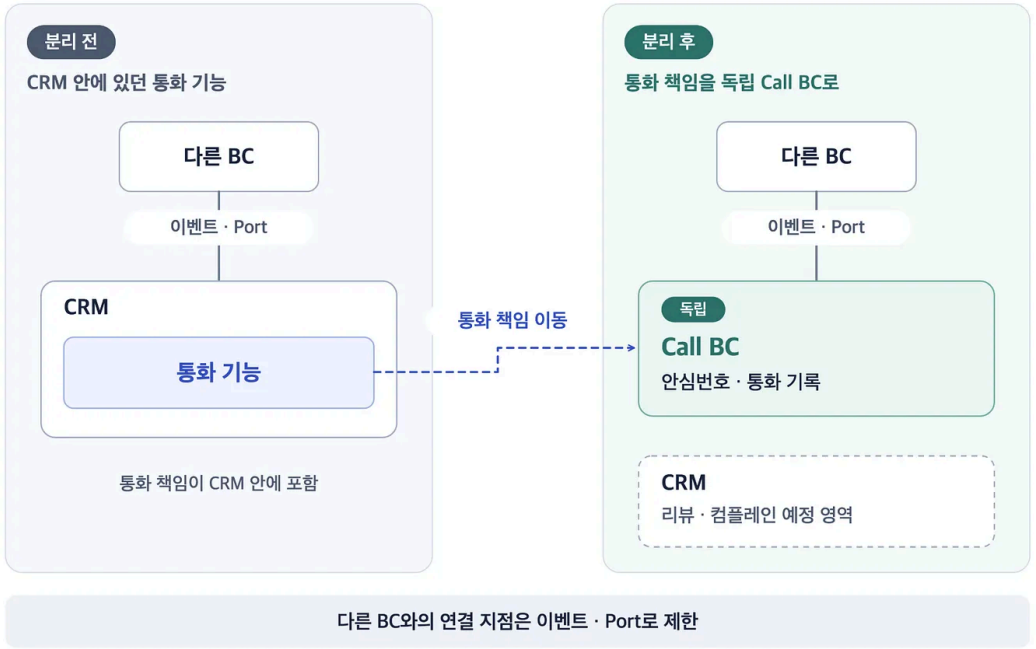
검증도 리뷰의 성격에 따라 보안, 아키텍처, 테스트처럼 역할과 체크리스트가 다른 에이전트에게 나눠 맡깁니다. 리뷰을 확정하거나 반박할 때는 파일과 코드 위치를 남기고, 근거가 부족하면 불확실한 상태로 둡니다.

최종 반영 여부는 사람이 판단합니다. 코드를 읽고 모르는 부분은 검색하거나 AI에게 질문합니다. 필요하면 AI에게 변경 내용을 그림으로 풀어 주는 문서를

만들게 해, 제안한 방식이 문제를 해결하는지, 프로젝트의 방향에 맞는지 살펴봅니다.

구현하면서 도메인 경계를 다시 나눈다

통화 기능은 처음에 리뷰와 컴플레인을 담당할 CRM BC에 두었습니다. 하지만 안심번호를 제공하는 BizCallMix를 연동하면서 통화 기능의 책임을 다시 살펴보았습니다. 안심번호 할당과 해제, 통화 기록 수신, '한 오더에 유효한 안심번호는 최대 하나'라는 규칙은 리뷰, 컴플레인 관리와 따로 다뤄야 했습니다. AI와 함께 책임과 경계를 논의한 뒤, 통화 기능을 Call BC로 분리했습니다.



통화 기능을 새 위치로 옮기며 114개 파일을 변경했습니다. 하지만 다른 BC와 맞춰야 할 부분이 이벤트와 Port에 모여 있어 내부 구현을 함께 옮기기 수월했습니다. ArchUnit으로 의존 관계를 확인하며 진행했고, 실제 작업은 하루가 채 걸리지 않았습니다.

처음 설계 때 확신하지 못했던 경계는 결국 한 번 바뀌었습니다. 경계를 처음부터 완벽하게 나누지 못해도, BC 사이의 연결을 이벤트와 Port로 좁혀 두면 바꾸는 비용은 작게 유지할 수 있었습니다.

서비스를 출시한 뒤, 같은 방식을 새 팀원에게 건넨다

서비스를 출시하고 나니, 처음 세웠던 목표를 돌아보게 됐습니다. 제가 쌓아 온 문서와 규칙, 세 단계의 작업 절차를 다음 사람도 활용할 수 있을까요? 최근 새 백엔드 팀원이 합류하면서 그 모습을 직접 볼 기회가 생겼습니다.

팀원에게 전달한 AI 사용법은 간단했습니다.

> 티켓을 확인하고 세 개의 스킬을 순서대로 쓰면 돼요. 하나는 설계, 하나는 구현, 하나는 코드리뷰예요.

별도로 설명하거나 도와야 할 부분은 크게 없었습니다. 팀원이 빠르게 적응해 기능을 만드는 모습을 보니 뿌듯했어요. 작업자의 경험과 판단은 여전히 필요하지만, 어디서 시작하고 무엇을 확인해야 할지 함께 참고할 기준은 남길 수 있었습니다.

남은 과제: 품질을 지키며 작업 속도를 높이기

그럼에도 아직 개선할 부분이 많습니다. 가장 중요한 것은 작업을 시작해 리뷰를 마칠 때까지 걸리는 시간을 줄이는 일입니다. AI가 코드를 빠르게 작성해도, 전체 작업이 끝날 때까지 오래 기다려야 한다면 그 속도를 충분히 활용하기 어렵습니다.

AI가 검증을 맡아도 결과를 얻기까지는 시간이 소요됩니다. 품질을 높이려다 에이전트의 검증 단계를 필요 이상으로 늘린 적이 있고, 작업의 난이도와 관계없이 최상위 모델만 사용해 본 적도 있습니다. 그 과정에서 AI의 검증 결과를 기

다리는 시간이 길어졌습니다. 사람이 그 결과를 확인하는 시간까지 포함하면 전체 작업 시간은 더 늘어났습니다.

앞으로는 각 AI 검증 단계가 실제 결함을 찾는 데 얼마나 도움이 되는지 확인하고, 작업의 난이도와 오류의 영향에 맞춰 모델과 검증 범위와 깊이를 조정하려 합니다. 결제 흐름처럼 오류의 영향이 큰 변경은 더 깊이 검증하고, 범위가 작고 되돌리기 쉬운 변경은 효과에 비해 시간이 많이 드는 추가 검증을 줄이는 식입니다. 품질을 유지하면서 전체 완료 시간이 줄어드는지, 재작업 시간까지 포함해 측정해 보려 합니다.

아정당 프로덕트팀은 함께 문제를 해결하고 성장할 다양한 직군의 동료를 찾고 있습니다.

관심있는 분들의 많은 지원을 기다립니다.

🙌 채용 공고 보기

Engineering + Ax +

Written by 아정당

24 followers · 2 following

Follow

Published in 아정당 기술 블로그

26 followers · Last published 6 days ago

Follow

365일 생활지원금을 지키는 아정당 프로덕트 팀, 우리가 만드는 기술과 개발 문화를 공유합니다.